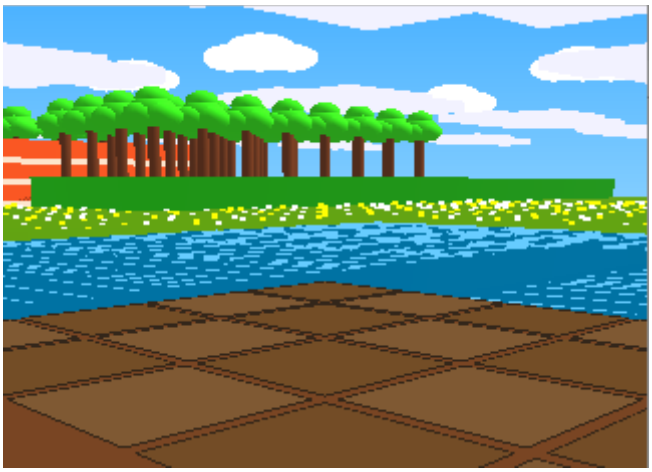
INTELIGENCIA ARTIFICIAL

E.T.S. de Ingenierías Informática y de Telecomunicación

Tutorial: Práctica 2 (Parte 2)

Agentes Reactivos/Deliberativos



DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN E INTELIGENCIA ARTIFICIAL

UNIVERSIDAD DE GRANADA

Curso 2025-2026

1. Introducción

El objetivo de la práctica es definir un comportamiento reactivo/deliberativo para un par de agentes cuya misión es desplazarse por un mapa. En este tutorial se pretende introducir al estudiante en la dinámica que debería seguir para la construcción de los niveles 2, 3 y 4, así también como de ayuda para los niveles 5 y 6 con la combinación de comportamientos reactivos y deliberativos.

En la primera parte de este tutorial se desarrollaron comportamientos reactivos. Esta segunda parte se sitúa como la continuación de la misma y tomando el software por donde se quedó. Entendemos que el estudiante habrá incluido más módulos para completar satisfactoriamente los niveles 0 y 1.

La mayoría de los estudiantes habrán realizado el tutorial 2 parte 1 y estarán familiarizados con las funciones que aquí se incluyen y que allí se definieron. Para aquellos estudiantes que no, recomendamos que vuelvan a mirar dicho tutorial.

Este tutorial 2 parte 2 se centra en realizar la implementación de un nivel ficticio de la práctica, construir el algoritmo de búsqueda en anchura para el agente Técnico, con la intención de servir de base para la construcción de los niveles 2, 3 y 4 reales de la práctica y ayudar a definir otros algoritmos que el estudiante pudiera querer emplear en los niveles 5 y 6.

2. Estructura general del agente

En la descripción de esta práctica se indica que hay dos agentes involucrados en los procesos de desplazamiento por el mapa, el agente Ingeniero y el agente Técnico, aunque en este tutorial trabajaremos solo con el agente Técnico. Además, se continuará con el diseño que ya se señaló en la parte 1 de definir un módulo independiente para resolver cada nivel de la práctica. Al ser un nivel ficticio el que vamos a resolver aquí (la búsqueda en anchura para el agente Técnico), crearemos un nuevo método para la clase ComportamientoTécnico que llamaremos:

Action ComportamientoTécnicoNivel_E(Sensores sensores)

que toma como parámetro de entrada el valor de los sensores del agente en el instante actual, y devuelve una acción. Esta forma de actuar se asemeja al comportamiento de un agente reactivo en el sentido que en cada iteración devuelve una única acción. Como en esta práctica parte se quieren evaluar comportamientos deliberativos necesitaremos adaptar el comportamiento de un agente deliberativo dentro de un esquema concreto. Lo que el agente hará será calcular un plan (en su versión más genérica de esta práctica), una secuencia de movimientos que permita trasladar al agente desde la casilla donde se encuentre a la casilla destino. Dicho plan se almacenará en una variable de estado. El método **ComportamientoTécnicoNivel_E** en cada iteración de la simulación irá mandando ejecutar la siguiente acción de ese plan hasta que el agente llegue a la casilla destino.

Vamos a implementar ese comportamiento básico sobre este método que ejecutará la siguiente acción del plan, si el plan existe. Si no existe, se calculará el plan. Para ello vamos a definir dos variables de estado: **plan** que almacenará el plan a ejecutar y que será de tipo **list<Action>** en la parte privada de la clase **ComportamientoTécnico** y **hayPlan** que nos dirá si ya se ha calculado un plan para llegar a la casilla objetivo que definiremos en la misma parte privada, pero en este caso su tipo asociado será **bool**. En el constructor de clase (en el que se usa para los niveles 2, 3, 4 y 5) la variable **hayPlan** se inicializará a **false**.

Adjunto al material de este tutorial se puede encontrar como quedaría el archivo "**tecnico.hpp**".

Se puede observar que se mantienen las variables de estado que se definieron para resolver el nivel 0, pero no aparecen aquellas que en vuestra versión particular de este fichero hayáis usado para resolver los dos primeros niveles de la práctica. Las modificaciones con respecto a como quedó tras el tutorial 1 son las siguientes:

```
196  private:
197    // ==========================================================================
198    // VARIABLES DE ESTADO (PUEDEN SER EXTENDIDAS POR EL ALUMNO)
199    // ==========================================================================
200
201
202  Action last_action;       // Almacena la última acción ejecutada
203  bool tiene_zapatillas;    // Indica si el agente tiene las zapatillas
204
205  //Nivel E        Chat Ctrl+L Edit Ctrl+I
206  bool hayPlan;            // Indica si hay una plan que ejecutar
207  list<Action> plan;       // Almacena el plan a realizar.
208  };
209
210  #endif
```

En la parte privada de la clase (líneas 205 a 207) se han incluido las nuevas variables de estado **plan** y **hayPlan**.

```
class ComportamientoTecnico : public Comportamiento {
public:
  // ========================================================================
  // CONSTRUCTORES
  // ========================================================================

  /**
   * @brief Constructor para niveles 0, 1 y 6 (sin mapa completo)
   * @param size Tamaño del mapa (si es 0, se inicializa más tarde)
   */
  ComportamientoTecnico(unsigned int size = 0) : Comportamiento(size) {
    // Inicializar Variables de Estado
    last_action = IDLE;
    tiene_zapatillas = false;
  }

  /**
   * @brief Constructor para niveles 2, 3, 4 y 5 (con mapa completo conocido)
   * @param mapaR Mapa de terreno conocido
   * @param mapaC Mapa de cotas conocido
   */
  ComportamientoTecnico(std::vector<std::vector<unsigned char>> mapaR,
                        std::vector<std::vector<unsigned char>> mapaC):
                        Comportamiento(mapaR, mapaC) {
    // Inicializar Variables de Estado
    hayPlan = false;
    tiene_zapatillas = false;
  }
```

- Se asignan a **false** en el constructor de clase para los niveles 2, 3, 4 y 5 tanto la variable de estado **hayPlan** como **tiene_zapatillas**.

```
110   * @param sensores Datos actuales de los sensores.
111   * @return Acción a realizar.
112   */
113    Action ComportamientoTecnicoNivel_5(Sensores sensores);
114
115  /**
116   * @brief Comportamiento del técnico para el Nivel 6.
117   * @param sensores Datos actuales de los sensores.
118   * @return Acción a realizar.
119   */
120    Action ComportamientoTecnicoNivel_6(Sensores sensores);
121
122
123    /**
124   * @brief Comportamiento del técnico para el Nivel E (especial tutorial parte 2).
125   * @param sensores Datos actuales de los sensores.
126   * @return Acción a realizar.
127   */
128    Action ComportamientoTecnicoNivel_E(Sensores sensores);      Chat Ctrl+L Edit Ctrl+I
129
```

- En línea 128 se añade el método ComportamientoTecnicoNivel_E que contendrá este nivel ficticio.

A partir de las variables anteriores, podemos plantear la siguiente estructura simple para el agente Técnico en el archivo "**tecnico.cpp**":

```
Action ComportamientoTécnico::ComportamientoTécnicoNivel_E(Sensores sensores){
	Action accion = IDLE;
	if (!hayPlan){
		// Invocar al método de búsqueda
		hayPlan = true;
	}
	if (hayPlan and plan.size()>0){
		accion = plan.front();
		plan.pop_front();
	}
	if (plan.size()== 0){
		hayPlan = false;
	}
	return accion;
}
```

En el esquema anterior, bastaría con poner un procedimiento que calcule un plan para el agente, y el resto del esquema lo ejecutará hasta que el plan se termine. Indicar en el código anterior, que ***Action*** es el tipo de dato que codifica las acciones posibles del agente y que ***IDLE*** es una constante del tipo ***Action***.

Imaginemos por ejemplo que deseamos que el agente se mueva sobre el terreno como si fuese un caballo de ajedrez, es decir, que avance dos casillas en la dirección en la que se encuentra, gire a la derecha (con dos acciones del tipo ***TURN_SR*** para completar un giro de 90 grados) y haga un avance más. Para realizar este tipo de movimiento, definimos la función ***AvanzaASaltosDeCaballo*** y la invocaremos en el método ***ComportamientoTécnicoNivel_E*** en el lugar de construcción del plan. A continuación se muestra como quedarían ambas funciones.

```
127
128 list<Action> AvanzaASaltosDeCaballo(){
129   list<Action> secuencia;
130   secuencia.push_back(WALK);
131   secuencia.push_back(WALK);
132   secuencia.push_back(TURN_SR);
133   secuencia.push_back(TURN_SR);
134   secuencia.push_back(WALK);
135   return secuencia;
136 }
137
138
139 /**
140  * @brief Comportamiento del técnico para el Nivel 2.
141  * @param sensores Datos actuales de los sensores.
142  * @return Acción a realizar.
143  */
144 Action ComportamientoTecnico::ComportamientoTecnicoNivel_E(Sensores sensores) {
145   Action accion = IDLE;
146   if (!hayPlan){
147     // Invocar al método de búsqueda
148     plan = AvanzaASaltosDeCaballo();
149     hayPlan = true;
150   }
151   if (hayPlan and plan.size()>0){
152     accion = plan.front();
153     plan.pop_front();
154   }
155   if (plan.size()==0){
156     hayPlan = false;
157   }
158   return accion;
159 }
```

Antes de compilar es necesario invocar a la función ***ComportamientoTécnicoNivel_E*** desde el método ***think***. Para ello utilizaremos el "case 3" del switch definido en ese método. Ahí, situaremos la llamada al método tal como se ve en la línea 22 de la siguiente imagen.

```
13 Action ComportamientoTecnico::think(Sensores sensores) {
14   Action accion = IDLE;
15
16
17   // Decisión del agente según el nivel
18   switch (sensores.nivel) {
19     case 0: accion = ComportamientoTecnicoNivel_0(sensores); break;
20     case 1: accion = ComportamientoTecnicoNivel_1(sensores); break;
21     case 2: accion = ComportamientoTecnicoNivel_2(sensores); break;
22     //case 3: accion = ComportamientoTecnicoNivel_3(sensores); break;
23     case 3: accion = ComportamientoTecnicoNivel_E(sensores); break;
24     case 4: accion = ComportamientoTecnicoNivel_4(sensores); break;
25     case 5: accion = ComportamientoTecnicoNivel_5(sensores); break;
26     case 6: accion = ComportamientoTecnicoNivel_6(sensores); break;
27   }
28
29   return accion;
30 }
```

Una vez hecho la anterior, compilamos y ponemos en un terminal

```
./practica2 -m mapas/mapa30.map -n 3 -i 7 7 2 -t 11 6 4 -O 12 5
```

y vamos pulsando al botón de *"paso"*, podremos observar como en un primer momento no hay plan, se incluye en la variable plan el movimiento del caballo y se aplica la primera acción de ese plan. En los siguientes 4 instantes se aplica la siguiente acción hasta que el plan se queda sin acciones. En el siguiente instante, se calcula de nuevo el plan y se ejecuta la primera acción. Este proceso se repite hasta que la simulación termine.

WALK	WALK	TURN_SR	TURN_SR	WALK

3. Construyendo el nivel ficticio E

El esquema anterior en el método ***ComportamientoTécnicoNivel_E*** es suficiente para llevar la ejecución de un plan cuando las condiciones del mundo son estáticas y se tiene información completa del mundo. Por tanto, el esquema anterior nos valdrá para los niveles 2 y 3 de la práctica y nos podremos centrar exclusivamente en implementar el algoritmo de búsqueda que nos piden e insertarlo en el lugar del ***AvanzaASaltosDeCaballo***.

En este nivel E (***especial***) que usamos para ilustrar como se deberían realizar los niveles 2 y 3 de la práctica vamos a implementar el algoritmo de búsqueda en anchura para llevar al agente Técnico a la casilla objetivo. Una descripción en pseudocódigo de este algoritmo podría ser algo como lo siguiente:

```
function BREADTH-1ST-SEARCH(problem)
begin
        current_node ← problem.INITIAL-STATE()
        frontier ← a FIFO queue
        explored ← an empty set
        frontier.insert(current_node);

        while (!frontier.empty() and !problem.Is_Solution(current_node)) do
        begin
                frontier.delete(current_node)
            explored.insert(current_node)
                for each action applicable to current_node do
                begin
                        child ← problem.apply(action, current_node)
                        if (problem.Is_Solution(child) then current_node = child
                        else if child.state() is not in explored or frontier then
                                frontier.insert(child)
                end
                if (!problem.Is_Solution(current_node) then
                        current_node ← frontier.next()
        end
        if (problem.Is_Solution(current_node)) then return SOLUTION(current_node)
        else return failure
end
```

Una descripción en lenguaje natural de ese proceso diría que se usan dos listas para recorrer el espacio de estados que se genera en la búsqueda, una lista con *alma* de COLA llamada *frontier* (que también se suele denominar ***frontera*** o ***abiertos*** en otras descripciones) que almacenará los estados pendientes de explorar y una lista con *alma* de CONJUNTO/SET llamada *explored* (o ***cerrados*** en otras descripciones) que almacena los estados que ya han sido explorados. La misión de la primera es mantener los estados aún no explorados en el mismo orden que se generaron en la

exploración, mientras que la misión de la segunda es asegurar que un estado ya explorado no se vuelve a explorar.

Se usa una variable ***current_node*** para almacenar el estado que se está evaluando en cada momento. En el instante inicial esa variable toma el valor del estado inicial del problema.

El resto del proceso es simple de describir; mientras la lista de nodos pendientes de explorar (*frontier*) no esté vacía y el estado actual no sea una solución, entonces se hace lo siguiente: se elimina el estado actual de la *frontier* y se inserta en la lista de estados explorados (*explored*). Para cada acción aplicable sobre el estado actual, se aplica y se genera un estado descendiente (*child*). Si ese estado es ya solución, la búsqueda termina. Si no lo es y ese estado no está en ninguna de las dos listas, se inserta en *frontier*.

El procedimiento solo se saldrá del ciclo o bien porque ***current_node*** es solución o porque *frontier* se vació. En el primer caso, se devolverá la solución asociada a ese estado. El segundo caso indica que después de explorar todos los estados que se podían generar desde el estado inicial del problema, ninguno de ellos satisfacía ser solución al problema.

3.1. Definición del concepto de estado y nodo

Ahora toca adaptar esta estrategia de búsqueda para nuestro problema y para ello debemos definir el concepto de ***estado***. Lo asociaremos al nivel 3 en lugar del 2, ya que es en este nivel el que espera que sea el Técnico el que llegue al destino. La definición que se haga establecerá la complejidad/tamaño del espacio de búsqueda y, por tanto, que soluciones son alcanzables y qué tiempo se necesitará para alcanzarlas.

El ***estado*** para este problema contiene la ***ubicación*** del agente Técnico (es decir su *fila*, su *columna* y su *orientación*) y también si está o no en posesión del objeto *Zapatillas*. La razón de incluir este último elemento se debe a que ese objeto hace transitables las casillas de tipo bosque ('B') y afecta a la solución que se pueda encontrar.

Así, en el fichero ***tecnico.hpp*** pasaré a definir el estado para este problema como:

```
struct EstadoT {
 ubicacion site;
 bool zapatillas;

 bool operator==(const EstadoT &st) const{
   return site == st.site and zapatillas == st.zapatillas;
 }
};
```

Se incluye en la definición el operador "==" para poder realizar comparaciones entre datos de tipo ***EstadoT***.

También es necesario definir el ***nodo*** que se utilizará en el algoritmo. Un ***nodo*** contiene la información de un estado junto con información adicional que se requiera para el uso del algoritmo

concreto de búsqueda. En este caso, en el nodo incluiremos un campo llamado ***secuencia*** en el que almacenaremos las acciones realizadas desde el estado inicial hasta el estado que se contempla en este nodo, y obviamente el estado asociado.

```
struct NodoT{
  EstadoT estado;
  list<Action> secuencia;

  bool operator==(const NodoT &node) const{
    return estado == node.estado;
  }
};
```

Se incluye en la definición el operador de igualdad para poder comparar datos de tipo ***NodoT***. Como se puede observar, se considera que dos datos de este tipo son iguales, si sus estados son iguales independientemente del contenido del campo secuencia.

3.2. Primera aproximación a la búsqueda en anchura

Fijado el concepto de ***estado*** y ***nodo***, pasamos a hacer una primera implementación del algoritmo de búsqueda en anchura intentando transcribir el pseudocódigo que se mostró anteriormente. A continuación se muestra la estructura general[1].

```
240  /**
241   * @brief Primera aproximación a la búsqueda en anchura
242   *
243   * @param inicio Estado Inicial de la búsqueda.
244   * @param final Estado Fianl de la búsqueda.
245   * @param terreno Matriz que contiene la información del terreno.
246   * @param altura Matriz que contiene la altura del mapa.
247
248   * @return La secuencia de acciones para llegar al estado final.
249   * @note Devuelve un plan vacío si no es posible encontrar un plan válido.
250   */
251  list<Action> ComportamientoTecnico::B_Anchura(const EstadoT &inicio, const EstadoT &final,
252                                                const vector<vector<unsigned char>> &terreno,
253                                                const vector<vector<unsigned char>> &altura){
254    NodoT current_node;
255    list<NodoT> frontier;
256    list<NodoT> explored;
257    list<Action> path;
258
259    current_node.estado = inicio;
260    frontier.push_back(current_node);
261    bool SolutionFound = (current_node.estado.site.f == final.site.f and current_node.estado.site.c == final.site.c);
262 >  while (!SolutionFound and !frontier.empty()){…
310    // Devuelvo el camino encontrado.
311    if (SolutionFound)
312      path = current_node.secuencia;
313
314    return path;
315  }
```

[1]Recordar incluir el método en la clase ***ComportamientoTécnico*** en el archivo "**tecnico.hpp**"

Describiremos poco a poco esta implementación empezando por los argumentos que se han considerado. Se puede observar que tiene 4 argumentos de entrada, el estado inicial llamado ***inicio*** que es de tipo ***EstadoT***, la casilla destino a donde debe llegar el agente jugador llamada ***final*** y que es de tipo ***EstadoT*** y por último los mapas sobre el que se realizará la búsqueda que se llama ***terreno*** el que contiene la información sobre los accidentes geográficos y ***altura*** con la información sobre la altura de las casillas. La salida de la función es la secuencia de acciones que llevan desde el estado inicial al final si es posible encontrar una solución o un plan vacío en otro caso.

Vamos a realizar la descripción del algoritmo en dos partes, por un lado, veremos la declaración de los datos que se usan junto con la estructura general del proceso de búsqueda, mientras que por el otro lado, veremos el cuerpo del ciclo que se dedica a la generación de los nuevos estados.

Vemos que los datos que se usan son los siguientes:

- ***current_node***: almacena el estado actual y que inicialmente toma el valor del parámetro inicio.
- ***frontier***: una lista que mantiene los nodos pendientes de explorar, inicialmente vacía.
- ***explored***: una lista que mantiene los nodos ya explorados, inicialmente vacía.
- ***path***: variable que devolverá la secuencia de acciones encontrada como solución.
- ***SolutionFound***: una variable lógica que determina si ya se ha encontrado un estado que satisface la condición de ser solución. Esta variable se inicializa mirando si ***current_note*** satisface la condición de ser solución, es decir, que el agente jugador esté en la fila y en la columna de la casilla destino.

Además de la declaración y antes de entrar en el ciclo principal del proceso de búsqueda (que va desde la línea 262 a la línea 310) lo único que se hace es meter el nodo actual en la lista de nodos pendientes de explorar que es equivalente a meter en dicha lista el estado inicial.

Del ciclo que se plantea en este proceso se sale por dos razones:

1. ***SolutionFound*** es cierto, lo que indica que se ha encontrado solución o
2. la lista de estados pendientes de explorar está vacía, en cuyo caso, se ha explorado todo el espacio alcanzable desde el estado inicial y no ha sido posible encontrar solución.

Por tanto, el valor de ***SolutionFound*** determina si fue posible encontrar una solución y es el valor que se devuelve.

Ahora pasamos a describir el cuerpo del ciclo del proceso de búsqueda y en él podemos distinguir 3 partes: (1) eliminar el estado actual de la lista de pendientes de explorar y meterlo en la lista de explorados [líneas 263 y 264], (2) generar todos los descendientes del estado actual [desde la línea 271 hasta la línea 302] y (3) tomar el siguiente valor de estado actual de la lista de estados pendientes de explorar [líneas 304 y 308].

```
262   while (!SolutionFound and !frontier.empty()){
263     frontier.pop_front();
264     explored.push_back(current_node);
265
266     // Compruebo si estoy en una casilla que da las zapatillas
267     if (terreno[current_node.estado.site.f][current_node.estado.site.c] == 'D'){
268       current_node.estado.zapatillas = true;
269     }
270
271     // Genero el hijo resultante de aplicar la acción WALK
272     NodoT child_Walk = current_node;
273     child_Walk.estado = applyT(WALK, current_node.estado, terreno, altura);
274     if (child_Walk.estado.site.f == final.site.f and child_Walk.estado.site.c == final.site.c){
275       // El hijo generado es solucion
276       child_Walk.secuencia.push_back(WALK);
277       current_node = child_Walk;
278       SolutionFound = true;
279     }
280     else if (!Find(child_Walk, frontier) and !Find(child_Walk, explored)){
281       // Se mete en la lista de frontier después de añadir a secuencia la acción
282       child_Walk.secuencia.push_back(WALK);
283       frontier.push_back(child_Walk);
284     }
285
286     if (!SolutionFound){
287       // El hijo resultante de aplicar la accion TURN_SR
288       NodoT child_TurnSR = current_node;
289       child_TurnSR.estado = applyT(TURN_SR, current_node.estado, terreno, altura);
290       if (!Find(child_TurnSR, frontier) and !Find(child_TurnSR, explored)){
291         child_TurnSR.secuencia.push_back(TURN_SR);
292         frontier.push_back(child_TurnSR);
293       }
294
295       // El hijo resultante de aplicar la accion TURN_SL
296       NodoT child_TurnSL = current_node;
297       child_TurnSL.estado = applyT(TURN_SL, current_node.estado, terreno, altura);
298       if (!Find(child_TurnSL, frontier) and !Find(child_TurnSL, explored)){
299         child_TurnSL.secuencia.push_back(TURN_SL);
300         frontier.push_back(child_TurnSL);
301       }
302     }
303
304     // Paso a evaluar el siguiente nodo en la lista "frontier"
305     if (!SolutionFound and !frontier.empty()){
306       current_node = frontier.front();
307       SolutionFound = (current_node.estado.site.f == final.site.f and current_node.estado.site.c
308     }
309   }
310   // Devuelvo el camino encontrado.
```

(1) En la parte de actualización de las listas se usan las operaciones primitivas para eliminar el primer elemento de una lista en el caso de ***frontier*** y para añadir al final de una lista en el caso de ***explored***.

(2) Se puede observar que la parte destinada a la generación de los estados descendientes del estado actual es lo que ocupa la mayor parte del código. Tres acciones que implican movimiento puede hacer el agente Técnico (**WALK**, **TURN_SR** y **TURN_SL**). El proceso que se hace para cada una de ellas es similar y para unificar la descripción nos centraremos en como se genera el descendiente de **TURN_SR** [líneas desde la 287 a la 293].

Primero, en línea 288 se crea una nueva variable de tipo NodoT que almacenará el resultado de aplicar la acción correspondiente al nodo actual y que en este caso se llama ***child_Turn_SR***. El cálculo del nuevo estado se hace a través de una función llamada ***applyT*** que describiremos más adelante, pero que aquí asumiremos que nos devuelve el nodo resultante de aplicar la acción.

Después (línea 290) se comprueba si el nuevo estado generado ya estaba en alguna de las dos listas (***frontier*** o ***explored***). Si no está en ninguna, entonces se añade como un nuevo nodo en la lista de estados pendientes de explorar después de añadir en la lista de acciones del nodo la acción realizada (línea 292). En esta implementación incluimos una función auxiliar llamada ***Find*** destinada a comprobar si un elemento está o no en la lista. Más adelante mostraremos la implementación de esta función.

El generar el siguiente nodo para la acción y comprobar si ya se había generado antes es común a todas las acciones que se pueden aplicar al estado actual aunque podemos observar que la acción **WALK** tiene algo más. Por la naturaleza de este problema, la acción **WALK** es la única que permite al agente cambiar de casilla, y solo cambiando de casilla es posible llegar al destino. Por esta razón, **y porque en la búsqueda en anchura, cuando se genera el primer estado que satisface las condiciones de ser solución es la solución al problema y el proceso de búsqueda debe terminar**, se incluye dicha verificación en la acción **WALK** [líneas de la 274 a la 279].

Por último, indicar que en la línea 286 se incluye una condición para que no se generen los estados descendientes de los giros si ya se generó un estado que es solución.

(3) En esta última parte del cuerpo del ciclo [línea 306] se toma el nuevo valor para current_node. Solo se toma un nuevo valor, si en la generación de los descendientes no se ha encontrado un estado que satisface las condiciones de ser solución y obviamente, si la lista de nodos pendientes de explorar (***frontier***) no está vacía.

Los procesos que permiten obtener el siguiente estado a partir del estado actual se encuentra encapsulado dentro de la función ***applyT***. Existirían muchas posibilidades de implementación de esta función.

3.2.1. Generando los siguientes estados (applyT)

En esta subsección se describe una posible implementación de la función ***applyT*** que genera a partir del estado actual el estado resultante de aplicar una acción. La implementación hace uso de dos funciones auxiliares. La primera de ellas es la función ***CasillaAccesibleTécnico***.

```
bool CasillaAccesibleTécnico(const EstadoT &st, const vector<vector<unsigned char>> &terreno, const vector<vector<unsigned char>> &altura){
    EstadoT next = NextCasillaTécnico(st);
    bool check1 = false, check2 = false, check3 = false;
    check1 = terreno[next.site.f][next.site.c] != 'P' and terreno[next.site.f][next.site.c] != 'M';
    check2 = terreno[next.site.f][next.site.c] != 'B' or (terreno[next.site.f][next.site.c] == 'B' and st.zapatillas);
    check3 = abs(altura[next.site.f][next.site.c] - altura[st.site.f][st.site.c]) <= 1;
    return check1 and check2 and check3;
}
```

Esta función toma tres argumentos de entrada, un EstadoT ***st*** y dos matrices bidimensionales ***terreno*** que indica los accidentes geográficos y ***altura*** que indica la altura de las casillas y devuelve si sería posible por parte del agente Técnico hacer una acción ***WALK*** desde esa posición.

La otra función de la que hace uso ***applyT*** es ***NextCasillaTécnico***. Esta función toma como dato de entrada un *estado* y devuelve *estado* en el que quedaría el agente después de hacer la acción.

```
EstadoT NextCasillaTécnico(const EstadoT &st){
    EstadoT siguiente = st;
    switch (st.site.brujula)
    {
    case norte:
            siguiente.site.f = st.site.f - 1;
            break;
    case noreste:
            siguiente.site.f = st.site.f - 1;
            siguiente.site.c = st.site.c + 1;
            break;
    case este:
            siguiente.site.c = st.site.c + 1;
            break;
    case sureste:
            siguiente.site.f = st.site.f + 1;
            siguiente.site.c = st.site.c + 1;
            break;
    case sur:
            siguiente.site.f = st.site.f + 1;
            break;
    case suroeste:
            siguiente.site.f = st.site.f + 1;
            siguiente.site.c = st.site.c - 1;
            break;
    case oeste:
            siguiente.site.c = st.site.c - 1;
            break;
    case noroeste:
            siguiente.site.f = st.site.f - 1;
            siguiente.site.c = st.site.c - 1;
    }
    return siguiente;
}
```

A partir de las dos funciones anteriores, planteamos ***applyT*** como una función con cuatro argumentos de entrada: la acción que se quiere realizar, el *estado actual* del agente y *terreno* y *altura*, mapas donde se representan los accidentes geográficos y la altura de las casillas. Devuelve el estado resultante tras aplicar la acción. Si no es aplicable la acción, devuelve como estado el mismo que recibió como argumento.

```
EstadoT applyT(Action accion, const EstadoT & st, const vector<vector<unsigned char>> &terreno, const vector<vector<unsigned char>> &altura){
	EstadoT next = st;
	switch(accion){
		case WALK:
			if (CasillaAccesibleTécnico(st,terreno,altura)){
				next = NextCasillaTécnico(st);
			}
		break;
		case TURN_SR:
			next.site.brujula = (Orientacion) ((next.site.brujula+1)%8);
			break;
		case TURN_SL:
			next.site.brujula = (Orientacion) ((next.site.brujula+7)%8);
			break;
	}
	return next;
}
```

La descripción de esta función es muy simple. En función de cada posible acción de las que se puede realizar (recordamos que el agente Técnico solo puede realizar tres acciones que implican movimiento) se cálcula el estado resultado de aplicar dicha acción sobre el estado ***st*** almacenándose el resultado final en la variable ***next*** que se inicializa con el valor de st al principio de la función.

3.2.2. Función para buscar en una lista (Find)

Para que quede definitivamente terminada la implementación de la primera implementación de la búsqueda en anchura queda por mostrar la codificación de la función ***Find***. Esta función tiene como argumentos de entrada un estado concreto ***st*** y una lista de estados y devuelve si está o no en lista.

```
bool Find (const NodoT & st, const list<NodoT> &lista){
	auto it = lista.begin();
	while (it != lista.end() and !((*it) == st)){
		it++;
	}
	return (it != lista.end());
}
```

Se puede observar que es una implementación de búsqueda secuencial sobre una lista y que tiene un orden de complejidad **O(n)**, siendo **n** el tamaño de la lista.

3.2.3. Función para visualizar el recorrido en el mapa (VisualizaPlan)

Ya viene incluida el método **VisualizaPlan** que se usa para proyectar como será el recorrido que ha planificado el agente sobre el mapa 2D. Toma como argumentos la ***ubicacion*** del estado inicial del ***plan*** y la lista de acciones a realizar. El recorrido se refleja en el mapa del simulador marcando con una línea amarilla las casillas que serán transitadas por el agente.

```
/**
* @brief Convierte un plan de acciones en una lista de casillas para
*       su visualización en el mapa gráfico.
* @param st    Estado de partida.
* @param plan  Lista de acciones del plan.
*/
void VisualizaPlan(const ubicacion &st, const list<Action> &plan);
```

3.2.4. Probando la primera implementación de la búsqueda en anchura

Una vez implementado el algoritmo de búsqueda podemos pasar a probar su funcionamiento ante algún problema de búsqueda. Para eso redefinimos el contenido del método **ComportamientoTécnicoNivel_E**.

```
/**
 * @brief Comportamiento del técnico para el Nivel E.
 * @param sensores Datos actuales de los sensores.
 * @return Acción a realizar.
 */
Action ComportamientoTecnico::ComportamientoTecnicoNivel_E(Sensores sensores) {
    Action accion = IDLE;
    if (!hayPlan){
            // Invocar al método de búsqueda
            EstadoT inicio, fin;
            inicio.site.f = sensores.posF;
            inicio.site.c = sensores.posC;
            inicio.site.brujula = sensores.rumbo;
            inicio.zapatillas = tiene_zapatillas;
            fin.site.f = sensores.BelPosF;
            fin.site.c = sensores.BelPosC;
            plan = B_Anchura(inicio, fin, mapaResultado, mapaCotas);
            VisualizaPlan(inicio.site,plan);
            hayPlan = plan.size() != 0 ;
    }
    if (hayPlan and plan.size()>0){
            accion = plan.front();
            plan.pop_front();
    }
    if (plan.size()== 0){
            hayPlan = false;
    }
    return accion;
}
```

Se puede observar que lo único que cambia con respecto a lo que aparecía anteriormente es la parte relacionada con la invocación al algoritmo de búsqueda (lo que hay en el bloque con el comentario // **Invocar al método de búsqueda**). En realidad, lo que se hace en ese trozo de código es definir e

inicializar el estado inicial y final que se pedirán en el algoritmo de búsqueda antes de llamar al propio algoritmo. El estado inicial se inicializa con los valores de fila, columna, orientación y si tiene zapatillas, mientras que en el estado final son solo necesarios los valores de fila y columna de la casilla objetivo.

Después de asignar los valores a inicio y fin, se hace la invocación al módulo ***B_Anchura*** pasándoles como argumentos el estado inicial y fin y los mapas *mapaResultado* y *mapaCotas* que contienen los accidentes geográficos y la altura de las casillas del mapa. El resultado se almacena en la variable de estado **plan**.

La siguiente sentencia hace que se vea el recorrido encontrado en el mapa del simulador y se asigna la variable de estado **hayPlan**. Esta variable de estado tendrá el valor true solo si se ha encontrado un plan.

Para probar el funcionamiento, compilamos y ejecutamos con el siguiente comando

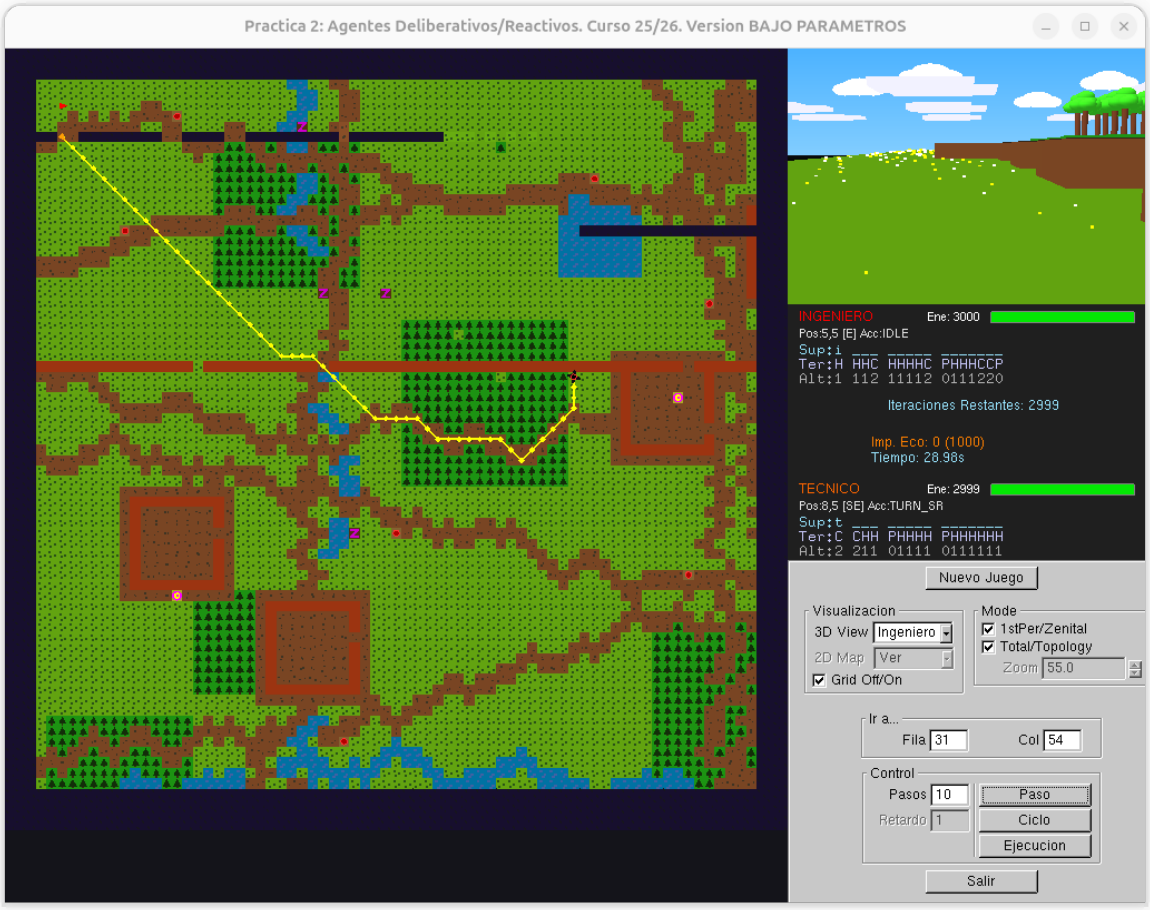
```
./practica2 -m mapas/mapa30.map -n 3 -i 5 5 2 -t 10 10 4 -O 12 5
```

Nos devuelve un plan que tiene 28 acciones. El recorrido se muestra con una línea amarilla sobre las casillas que deberá recorrer el agente para llegar a la casilla de la *Belkanita*.

Si se va pulsando el botón "Paso", se ve como el agente va recorriendo las casillas marcadas hasta llegar al objetivo. Una vez en el objetivo, la simulación termina.

Repetimos la ejecución pero ahora con la siguiente llamada[2]

```
./practica2 -m mapas/mapa75.map -n 3 -i 5 5 2 -t 8 5 2 -O 31 54
```



En este caso, **tarda unos 30 segundos** en conseguir el plan (este valor de tiempo depende del ordenador donde se ha ejecutado y por tanto puede darte un valor distinto). El plan está compuesto por 62 acciones.

[2]Ten paciencia. La ejecución tarda, no se "colgó". Si tarda más de una hora, mmmm, lo mismo sí se colgó :)

3.3. Segunda versión de la búsqueda en anchura: acelerando la búsqueda

En esta segunda versión, modificaremos ligeramente el método de búsqueda para mejorar el tiempo de respuesta del algoritmo. Las modificaciones afectarán fundamentalmente a las estructuras de datos que utilizamos para almacenar las dos listas. La lista de nodos pendientes de explorar (frontier) está implementada siguiendo el **TDA** lista. Las operaciones que requiere el algoritmo de búsqueda sobre esta lista son 4: *insertar*, *leer el primero*, *sacar el primero* y *buscar*. Las 3 primeras operaciones son muy eficientes y se podrían haber hecho usando un TDA cola (queue), pero en la implementación de queue en la ***STL*** de C++ no tiene un operador para buscar y por eso razón se usó la TDA de ***lista*** usándola como si fuera una cola (el primero en entrar es el primero en salir).

La búsqueda de elementos en una lista tiene orden de complejidad **O(n)** como ya hemos referido varias veces a lo largo de este tutorial y no resulta un mecanismo muy eficiente si se requiere hacerlo muchas veces y si las listas son muy grandes como es el caso de este algoritmo de búsqueda. Se requiere buscar en *frontier* cada vez que se genera un nuevo estado.

En esta versión del algoritmo de búsqueda en anchura que aquí proponemos, vamos a eliminar la búsqueda de nodos en *frontier*, así que mantenemos el tipo *list* asociado a esta lista y todas las búsquedas de nodos se harán sobre la lista *explored*.

Si todas las búsquedas las haremos sobre explored, se deberá usar una estructura de datos eficiente para realizar esta operación en dicha lista. Vamos a optar por usar el **TDA *set*** para su implementación. Sobre esta estructura de datos, las operaciones de búsqueda e inserción son de ***O(log$_2$(n))*** que son justo las operaciones que el algoritmo de búsqueda requiere.

El tipo de dato ***set*** de la **STL** de **C++** requiere definir un operador de orden entre los elementos que forman parte de la lista, ya que en la base de su implementación hay un árbol binario de búsqueda. Por esa razón, cuando definimos el tipo NodoT, definimos el operador *menor estricto*. Es este operador el que usará el tipo set para ordenar los elementos dentro de la estructura. Modificaremos la definción del ***struct NodoT*** en el archivo "**tecnico.hpp**" para que quede como se indica a continuación:

```
struct NodoT{
  EstadoT estado;
  list<Action> secuencia;

  bool operator==(const NodoT &node) const{
    return estado == node.estado;
  }

  bool operator<(const NodoT &node) const{
    if (estado.site.f < node.estado.site.f) return true;
    else if (estado.site.f == node.estado.site.f and estado.site.c < node.estado.site.c) return true;
    else if (estado.site.f == node.estado.site.f and estado.site.c == node.estado.site.c and estado.site.brujula <
node.estado.site.brujula) return true;
    else if (estado.site.f == node.estado.site.f and estado.site.c == node.estado.site.c and estado.site.brujula ==
node.estado.site.brujula and estado.zapatillas < node.estado.zapatillas) return true;
    else return false;
  }
};
```

Aunque debe tener estructura de relación de orden por los requerimientos del ***set***, desde el punto de vista del papel que juega dentro del algoritmo de búsqueda en realidad se puede ver como un operador de igualdad, y con esa visión se podría leer como: *dos nodos son distintos si la fila del agente de un nodo es menor que la fila del agente del otro nodo. También son distintos si siendo lo anterior igual, la columna del agente de un nodo es menor que la columna del agente del otro nodo. Siendo las filas y las columnas iguales, será distinto si las orientaciones no coinciden. Por último, son distintos si siendo las filas, las columnas y las orientaciones iguales para los dos nodos, uno tiene las zapatillas y el otro no. Si no pasa lo que se ha descrito antes, los dos nodos son iguales*.

Bueno, ya hemos tomado la decisión sobre los tipos de datos de las listas, ***frontier*** seguirá siendo una lista de NodoT sobre el que no se hará ninguna búsqueda y ***explored*** será un ***set*** de NodoT y solo sobre él se realizarán todas las búsquedas para encontrar los nodos repetidos.

```
320  /**
321   * @brief Segundoa aproximación a la búsqueda en anchura
322   *
323   * @param inicio Estado Inicial de la búsqueda.
324   * @param final Estado Fianl de la búsqueda.
325   * @param terreno Matriz que contiene la información del terreno.
326   * @param altura Matriz que contiene la altura del mapa.
327
328   * @return La secuencia de acciones para llegar al estado final.
329   * @note Devuelve un plan vacío si no es posible encontrar un plan válido.
330   * @note Explored pasa a ser implementado mediante un "set" en lugar de un "list"
331   */
332  list<Action> ComportamientoTecnico::B_Anchura_V2(const EstadoT &inicio, const EstadoT &final,
333                                                   const vector<vector<unsigned char>> &terreno,
334                                                   const vector<vector<unsigned char>> &altura){
335    NodoT current_node;
336    list<NodoT> frontier;
337    set<NodoT> explored;
338    list<Action> path;
339
340    current_node.estado = inicio;
341    frontier.push_back(current_node);
342    bool SolutionFound = (current_node.estado.site.f == final.site.f and current_node.estado.site.c == final.site.c);
343 >  while (!SolutionFound and !frontier.empty()){ ⋯
394
395    // Devuelvo el camino encontrado.
396    if (SolutionFound)
397      path = current_node.secuencia;
398
399    return path;
400  }
```

Mirando la estructura general podemos observar que solo hay un cambio con respecto a la versión anterior y es en la línea 428 donde ahora ***explored***, la lista de nodos ya explorados, se define de tipo ***set***.

```
343   while (!SolutionFound and !frontier.empty()){
344     frontier.pop_front();
345     explored.insert(current_node);
346
347     // Compruebo si estoy en una casilla que da las zapatillas
348     if (terreno[current_node.estado.site.f][current_node.estado.site.c] == 'D'){
349       current_node.estado.zapatillas = true;
350     }
351
352     // Genero el hijo resultante de aplicar la acción WALK
353     NodoT child_Walk = current_node;
354     child_Walk.estado = applyT(WALK, current_node.estado, terreno, altura);
355     if (child_Walk.estado.site.f == final.site.f and child_Walk.estado.site.c == final.site.c){
356       // El hijo generado es solucion
357       child_Walk.secuencia.push_back(WALK);
358       current_node = child_Walk;
359       SolutionFound = true;
360     }
361     else if (explored.find(child_Walk) == explored.end()){
362       // Se mete en la lista de frontier después de añadir a secuencia la acción
363       child_Walk.secuencia.push_back(WALK);
364       frontier.push_back(child_Walk);
365     }
366
367     if (!SolutionFound){
368       // El hijo resultante de aplicar la accion TURN_SR
369       NodoT child_TurnSR = current_node;
370       child_TurnSR.estado = applyT(TURN_SR, current_node.estado, terreno, altura);
371       if (explored.find(child_TurnSR) == explored.end()){
372         child_TurnSR.secuencia.push_back(TURN_SR);
373         frontier.push_back(child_TurnSR);
374       }
375
376       // El hijo resultante de aplicar la accion TURN_SL
377       NodoT child_TurnSL = current_node;
378       child_TurnSL.estado = applyT(TURN_SL, current_node.estado, terreno, altura);
379       if (explored.find(child_TurnSL) == explored.end()){
380         child_TurnSL.secuencia.push_back(TURN_SL);
381         frontier.push_back(child_TurnSL);
382       }
383     }
384
385     // Paso a evaluar el siguiente nodo en la lista "frontier"
386     if (!SolutionFound and !frontier.empty()){
387       current_node = frontier.front();
388       while (explored.find(current_node) != explored.end() and !frontier.empty()){
389         frontier.pop_front();
390         current_node = frontier.front();
391       }
392     }
393   }
394
395   // Devuelvo el camino encontrado.
```

Las cosas que cambian dentro del ciclo principal del algoritmo de búsqueda en anchura [desde la línea 343 a la 393] son:

- la operación de añadir un nuevo nodo a la explored (antes al ser una lista se usaba el método push_back y ahora al ser un set se usa el método insert) en la línea 345.
- no se busca en las dos listas sino que solo se busca en la lista de explored y eso se expresa en las líneas 361, 371 y 379.

- el proceso para elegir el siguiente nodo actual [líneas de la 385 a 392]. Para entender esto necesitamos conocer que implica no buscar nodos en frontier.

Explicación del cambio en la elección del siguiente nodo a evaluar.

- Antes de esta modificación **no había nodos repetidos ni en frontier**.
- Ahora **varias instancias de un mismo nodo pueden aparecer múltiples veces en frontier**.
- En principio, esto no implicaría un comportamiento distinto al algoritmo de búsqueda en anchura siempre y cuando antes de tomar un nodo como nodo actual, se verifique que no es un nodo que ya se haya explorado.

Pues justo eso es lo que hace el trozo de código de las líneas anteriormente referidas, van tomando nodos de frontier en el mismo orden en el que fueron añadidos a la lista hasta encontrar el primero de ellos que verifica no estar ya en explored. Con esta modificación, lo que hacemos es permitir que la lista frontier ocupe potencialmente mucho más espacio a cambio de hacer menos procesos de búsqueda con la intención de reducir el tiempo necesario para encontrar una solución.

Vamos a probar el funcionamiento de esta tercera versión, y para ello sustituimos en la función que invoca al algoritmo de búsqueda en el método **ComportamientoTécnicoNivel_E** la línea

```
hayPlan = B_Anchura(inicio, fin, mapaResultado, mapaCotas);
```

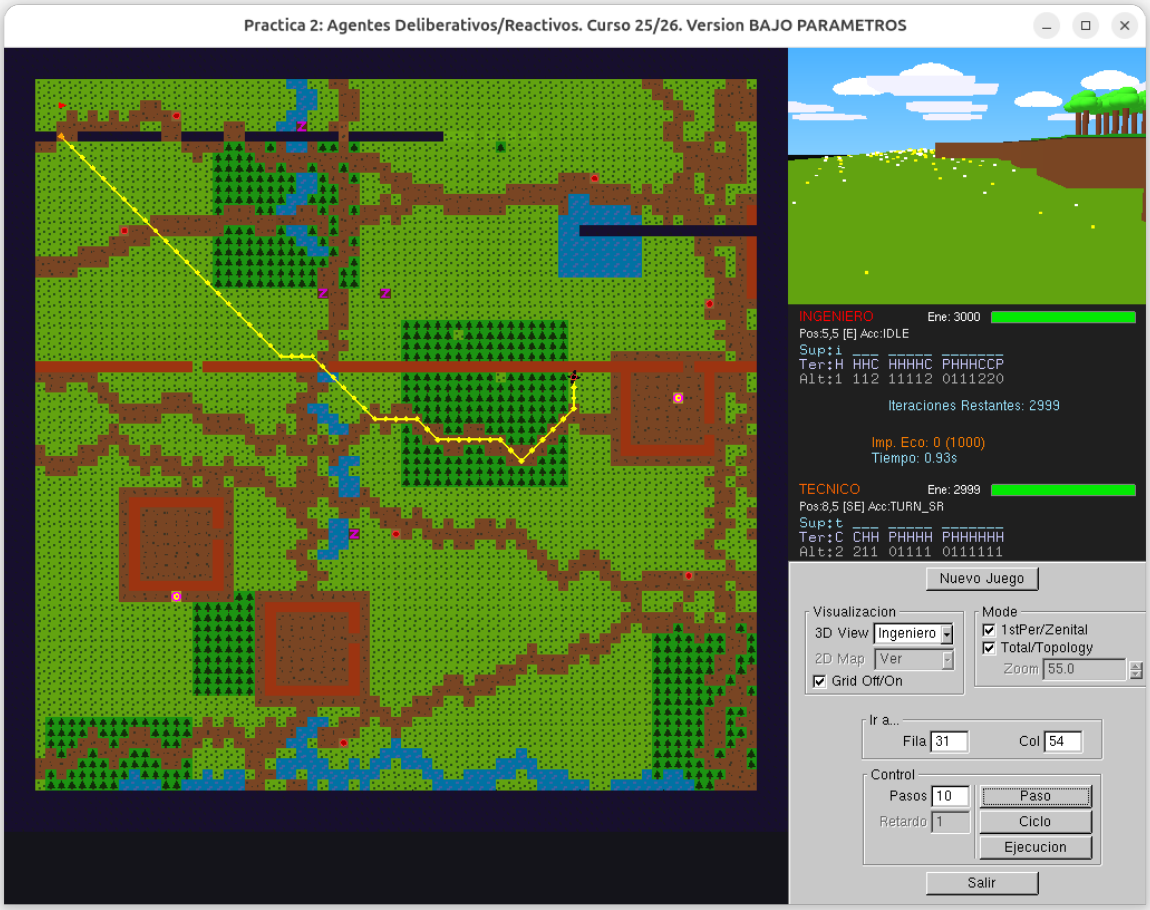
por

```
hayPlan = B_Anchura_V2(inicio, fin, mapaResultado, mapaCotas);
```

compilamos y ejecutamos

```
./practica2 -m mapas/mapa75.map -n 3 -i 5 5 2 -t 8 5 2 -O 31 54
```

obtenemos



Se puede observar que el tiempo que se ha consumido para el agente Técnico es de menos de un segundo (0.93 seg) frente a los cerca de 30 segundos que necesitó la versión anterior. El plan resultante es el mismo y contiene 62 acciones.

4. Comentarios sobre Niveles 2 y 3

En esta parte del tutorial se ha planteado la solución a un nivel ficticio de una dificultad semejante a los niveles 2 y 3 de la práctica 2. Muchas de las funciones aquí definidas serán de utilidad directa para resolver esos niveles y otras deberán cambiarse para adaptarse a las peculiaridades de cada uno de los algoritmos que se pide y también a las peculiaridades de los agentes. Indicar que la estructura general entre los algoritmos solicitados en la práctica y el que se ha desarrollado aquí son muy parecidos. Aunque son muy parecidos, el estudiante tiene que tener en cuenta que hay dos grandes diferencias con los algoritmos que debe implementar en la práctica:

1. La gestión de la lista ***frontier***. Aquí, en la búsqueda en anchura, se comporta como una cola. En los otros algoritmos no es así y deberá seleccionarse una estructura de datos apropiada para hacer eficiente su gestión.
2. La condición de finalización de la búsqueda. En anchura, la búsqueda termina cuando se genera un hijo que satisface las condiciones de alguno de los estados finales. Por esa razón, en la implementación, después de hacer **WALK** se miraba si se había llegado a la casilla objetivo. En los algoritmos solicitados, **la primera solución puede no ser la mejor solución** y por tanto la condición de salida es distinta ya que se piden soluciones óptimas en consumo de energía o en impacto ecológico.

Con el fin de ayudar al estudiante en el desarrollo de este tutorial, la siguiente imagen indica el orden en el que se han incluyendo los distintos módulos en el archivo "**tecnico.cpp**".

```
120   // ======================================== Nivel Especial del Tutorial 2p2 ==========================
121 > /**…
126 > void AnularMatrizA(vector<vector<unsigned char>> &m)…
136
137 > /**…
143 > void ComportamientoAuxiliar::VisualizaPlan(const EstadoA &st, const list<Action> &plan)…
194
195 > /**…
201 > EstadoA NextCasillaAuxiliar(const EstadoA &st){…
235
236 > /**…
245 > bool CasillaAccesibleAuxiliar(const EstadoA &st, const vector<vector<unsigned char>> &terreno, const vector<v
253
254 > /**…
264 > EstadoA applyA(Action accion, const EstadoA & st, const vector<vector<unsigned char>> &terreno, const vector<
278
279 > /**…
287 > bool Find (const NodoA & st, const list<NodoA> &lista){…
294
295 > /**…
301 > void ComportamientoAuxiliar::PintaPlan(const list<Action> &plan, bool zap)…
344
345 > /**…
355   list<Action> ComportamientoAuxiliar::AnchuraAuxiliar(const EstadoA &inicio, const EstadoA &final,
356 >                                                      const vector<vector<unsigned char>> &terreno, const vector<vector<unsign
413
414 > /**…
424   list<Action> ComportamientoAuxiliar::AnchuraAuxiliar_V2(const EstadoA &inicio, const EstadoA &final,
425 >                                                      const vector<vector<unsigned char>> &terreno, const vector<vector<unsign
485
486 > /**…
491 > list<Action> AvanzaASaltosDeCaballo(){…
500
501 > /**…
507 > Action ComportamientoAuxiliar::ComportamientoAuxiliarNivel_E(Sensores sensores){…
```

Una estructura semejante, en cuanto a las funciones definidas aquí, pero adaptadas para al otro de los agentes, debe realizar el estudiante. Entre esas adaptaciones está la definición de las estructuras EstadoI y NodoI que definirán el concepto de estado y de nodo del agente Ingeniero.

Para la realización del nivel 3 real de la práctica que puede interferir lo realizado en este nivel E descrito en este tutorial. El estudiante puede decidir poner un nombre distinto al nuevo concepto de estado o nodo del nuevo algoritmo o bien, redifinir EstadoT y nodoT para aplicarlo a su algoritmo y eliminar los métodos ***B_Anchura*** y ***B_Anchura_V2*** de los archivos. Estos métodos son un apoyo para el estudiante, pero no tienen porqué aparecer en la versión final de la práctica que se entregará.

5. El Nivel 4

Al nivel 4 de la práctica se le puede ver como un nivel especial. En esencia, es muy parecido a los niveles 2 y 3, ya que lo que fundamental que se requiere es la definición de otro algoritmo de búsqueda, pero tiene algunas peculiaridades:

1) mientras en los niveles 2 y 3, además de la elaboración plan, se pide que los agentes apliquen el plan sobre el mundo para que se compruebe fehacientemente la validez del mismo, en este nivel sólo se pide la elaboración del plan. Por esa razón, este nivel se resuelve usando solo un instante de simulación. En el instante inicial, el agente Ingeniero que es el encargado de realizar la planificación de la red de tuberías toma conciencia de como es el mundo, construye el plan, y lo manda al entorno para evaluar su validez.

2) el plan elaborado no implica desplazamiento concreto de ningún agente, sino que indica que casillas se verán involucradas en el desarrollo de la red de tuberías. Esto implica que aquí el ***plan*** no será de tipo ***list<Action>,*** sino de tipo ***list<Paso>***.

```
struct Paso {
  int fil;
  int col;
  int op; // -1 bajar la casilla, 0 dejar como está, 1 subir la casilla
};
```

Paso es una estructura con 3 campos que indican la ***fil***, la ***col*** haciendo referencia a una casilla del mapa por donde hay que instalar una tubería y **op**, que tomará el valor -1 si hay que hacer una acción **DIG** sobre ella antes de instalar la tubería, el valor +1 si hay que hacer una operación **RAISE** y 0 si no se debe cambiar su altura.

Un plan válido ***list<Paso>*** debe empezar con la casilla de la *Belkanita* y debe terminar con una casilla de tipo 'U', planta de tratamiento de residuos. No es la única condición. Debe respetar también la adyacencia ortogonal (solo están permitidas construcciones de tuberías con conexiones norte-sur o este-oeste, nunca diagonales) y el principio de desplazamiento por gravedad del fluido, es decir, la altura de la casilla que está en la posición *i* de una ***list<Paso>*** debe ser igual o como mucho mayor en una unidad a la que ocupa la posición *i+1*.

3) el algoritmo que se pide implementar debe optimizar un criterio múltiple, debe minimizar el número de operaciones **INSTALL** pero siempre respetando que el plan global no supere el impacto ecológico máximo permitido.

4) la validez del trazado se verifica haciendo una llamada a

```
void VisualizaRedTuberias(const list<Paso> &plan);
```

donde el plan calculado por el Ingeniero se añade como parámetro. El entorno verifica si el plan el válido (respeta las restricciones impuestas) y si es así, la simulación termina.

```
/**
 * @brief Convierte un plan de tubería en la lista de casillas usada
```

```
*       por el sistema de visualización.
* @param st    Estado de partida (no utilizado directamente).
* @param plan  Lista de pasos del plan de tubería.
*/
 void VisualizaRedTuberias(const list<Paso> &plan);
```

6. Los Niveles 5 y 6

Por último, una pequeña mención a los últimos niveles de la práctica. Los niveles del 0 al 4 están orientados a entender que las soluciones que se pueden encontrar para los niveles 5 y 6 pueden estar formada en componer estos comportamientos definidos previamente.

Decir que no hay una única manera para resolver cada uno de estos niveles y que hay diferentes estrategias que podéis explorar para ver cual de ellas tiene un mejor comportamiento medio.

En la resolución de estos niveles, y en las partes que vosotros estiméis oportuno, podéis invocar directamente al nivel que lo resuelve, pero también podréis definir nuevos comportamientos semejantes a los definidos en los niveles anteriores que creáis que van a tener un mejor desempeño.